

Computer Vision:

Homework-5

Bilal Ahmed
ahmedb@purdue.edu

1 Objective

In this assignment, the objective is to create panoramic images by stitching together individual images. This process involves utilizing linear least squares for initial homography estimation while incorporating outlier rejection through the RANSAC (RANdom SAmple Consensus) algorithm. Furthermore, the estimated homography will be refined using non-linear least squares optimization via the Levenberg-Marquardt (LM) algorithm. Through these methods, we aim to achieve a seamless and coherent panorama from the individual image inputs.

For RANSAC, the following settings were used:

- $n = 5$: the number of points used for estimating the homography by solving the homogeneous linear least squares problem using singular value decomposition (SVD).
- $\sigma = 5$: The noise in the projected pixel values is modeled as a Gaussian distribution with a standard deviation of σ . A projected point with a geometric distance, as described below, exceeding 3σ is classified as an outlier.
- ϵ : the probability that a randomly chosen correspondence is an outlier was chosen adaptively, starting from a worst-case assumption of 0.9 and updating this value each time the inlier set is larger than the previous set.
- $p = 0.99$: the desired probability that at least one trial in N contains no outliers.
- $N = \frac{\ln(1-p)}{\ln(1-(1-\epsilon)^n)}$: the equation governing the number of iterations required for the RANSAC algorithm.
- **Normalized Homography Estimation**: To compute homography in a stable manner, normalized coordinates were used with pre-conditioning matrices. This provides a more stable homography matrix, calculated as follows:

$$H = T'^{-1}\tilde{H}T$$

where $\tilde{H}$ is the estimated homography from normalized points, T is the normalization transformation matrix for the source image, and T' is the normalization transformation matrix for the destination image, with T applied in the homography calculation.

- **Geometric Distance**: Geometric distance was computed in both domain space and range space using reprojection, defined as:

$$d(p, p') = \|x' - Hx\|^2 + \|x - H^{-1}x'\|^2$$

where:

- x' is the point in the range space,
- Hx represents the predicted point in the range space after applying the homography H ,
- x is the original point in the domain space,
- $H^{-1}x'$ represents the predicted point in the domain space after applying the inverse homography H^{-1} .

1.1 Non-linear Least Squares using Levenberg-Marquardt

In this section, we utilize the Levenberg-Marquardt algorithm to refine the homography estimation obtained from the RANSAC method. This algorithm combines the benefits of both the gradient descent and the Gauss-Newton methods, allowing for improved convergence properties and robustness in the optimization process. The objective is to minimize the reprojection error defined as follows:

$$C(H) = \|X - f(H)\|^2$$

where:

- $C(H)$ is the error function to be minimized,
- X is the vector of all original key points stacked on top of each other, resulting in a vector of size $2m$ for m points,
- $f(H)$ represents the non-linear function of H that gives the predicted coordinates corresponding to X .

The Levenberg-Marquardt algorithm iteratively updates the homography H to find a set of parameters that best fit the observed data while minimizing the reprojection error.

For the Levenberg-Marquardt (LM) algorithm, selecting an appropriate initial guess is crucial, as it must be within the vicinity of the optimal solution. In this implementation, I utilized the output from the RANSAC (Random Sample Consensus) algorithm, which estimates the homography through linear least squares, as the initial guess. This initial estimation is then further refined using the LM algorithm to achieve improved accuracy in the homography parameters.



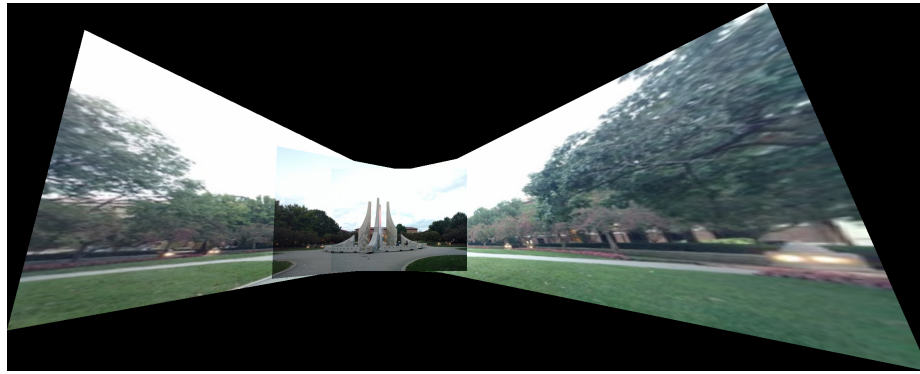
Figure 1: Five Images of fountain (task-1)



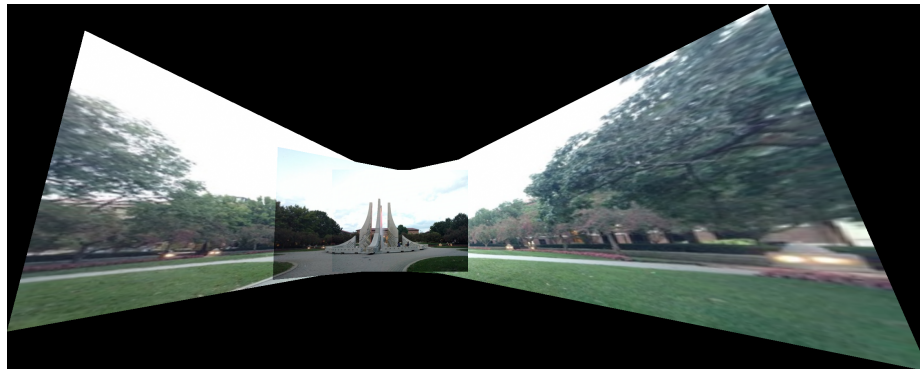
Figure 2: Pairwise images with inliers (green) and outliers (red).

Table 1: Geometric distances for various image pairs with and without landmark matching (LM)

image pair	Distance without LM	Distance with LM
1-2	2.377	2.376
2-3	16.078	16.078
3-4	19.675	19.672
4-5	4.644	4.629
11-12	17.012	16.990
12-13	20.840	20.811
13-14	33.596	33.596
14-15	40.549	40.509



(a) without LM.



(b) with LM.

Figure 3: Panoramic view of images of fountain (set-1).

2 Theoretical Questions

1. Conceptually speaking, how do we differentiate between the inliers and the outliers when using RANSAC for solving the homography estimation problem using the interest points extracted from two different photos of the same scene?

Answer: We distinguish between inliers (true but noisy correspondences) and outliers (false correspondences) by evaluating the geometric distance between the true coordinate values and the projected values. Specifically, we model the projected coordinates to have a Gaussian distribution with a standard deviation of σ . Key points that have a geometric distance greater than 3σ are considered outliers.

A robust way of computing this geometric error in both the domain and range spaces is given by the following expression:

$$d(p, p') = \|x' - Hx\|^2 + \|x - H^{-1}x'\|^2$$

where:

- x' is the point in the range space,
- Hx represents the predicted point in the range space after applying the homography H ,
- x is the original point in the domain space,
- $H^{-1}x'$ represents the predicted point in the domain space after applying the inverse homography H^{-1} .

2. Explain how the Levenberg-Marquardt (LM) algorithm combines the best of GD and GN to give us a method that is reasonably fast and numerically stable at the same time.

The Levenberg-Marquardt algorithm combines the benefits of gradient descent and the Gauss-Newton method by utilizing a damping factor, μ . This damping factor determines the algorithm's behavior:

- When μ is large, the algorithm behaves more like gradient descent (GD), as the damping term dominates the optimization process.
- Conversely, when μ is small, the algorithm resembles the Gauss-Newton (GN) method, since the Jacobian transpose term $J_f^T J_f$ becomes the dominant factor.

The update rule for the LM algorithm is given by:

$$\Delta \mathbf{p} = (J_f^T J_f + \mu I)^{-1} J_f^T \mathcal{E}$$

where:

- $\Delta \mathbf{p}$ is the parameter update,
- J_f is the Jacobian matrix of the non-linear function,
- $\mathcal{E}$ is the vector of residuals,
- μ is the damping factor,
- I is the identity matrix.

By adjusting μ throughout the optimization process, the algorithm can effectively balance between the stability of GD and the faster convergence of GN.

3 Results

Figure 1 shows five individual images used for creating the panorama in Task ID 1.

Figure 4 presents five individual images for Task ID 2.

Figures 2 and 5 illustrate the pairwise matches images with inliers shown in green and outliers shown in red.

Figure 3 shows panoramic views obtained by stitching the images of fountain (task-1), with top figure created without using LM and bottom image using with LM.

Table 1 summarizes the geometric distances for various image pairs, comparing the distances calculated without and with the use of the Levenberg-Marquardt (LM) optimization. The results demonstrate the improvement in distance measurements achieved by applying LM.

Figure 6 shows panoramic views obtained by stitching the images of Wang Hall (task-2), with top figure created without using LM and bottom image using with LM.



Figure 4: Five Images of Wang Hall (task-2)

Python Code

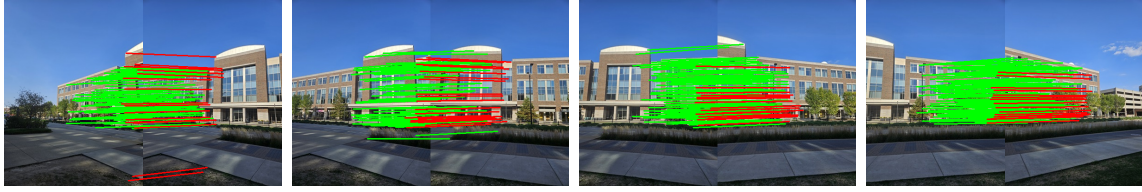
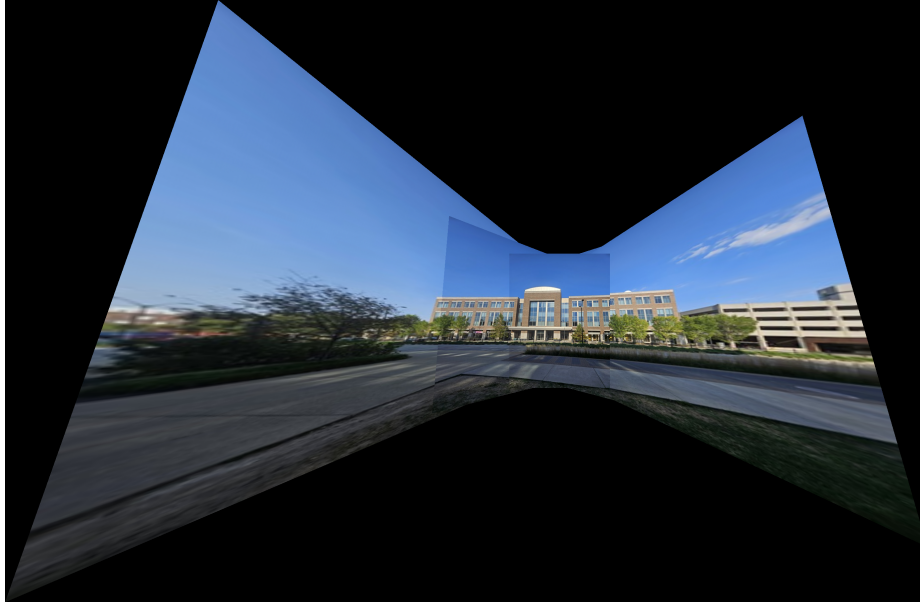
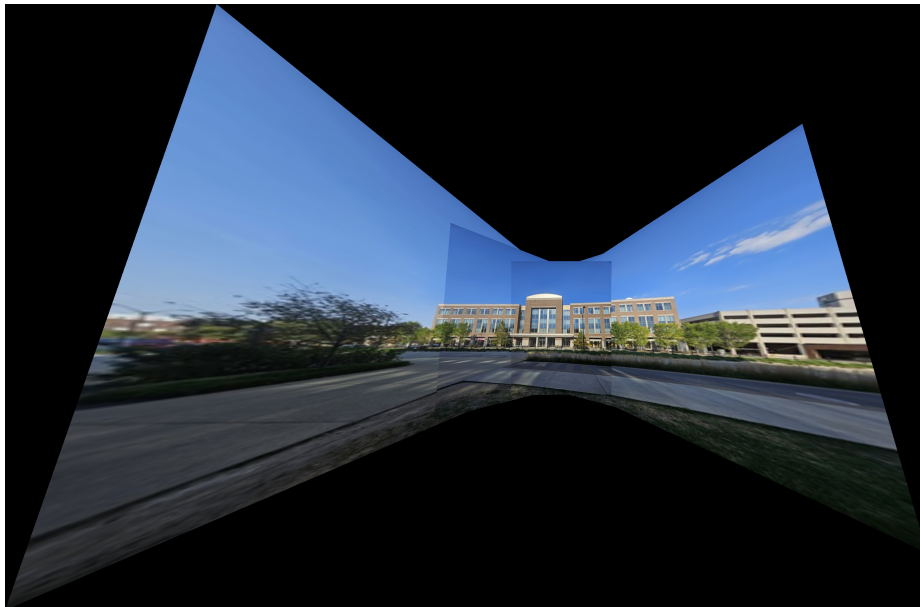


Figure 5: Pairwise images with inliers (green) and outliers (red).



(a) without LM.



(b) with LM.

Figure 6: Panoramic view of images of fountain (set-2).

```
1 import os
2 import pandas as pd
3 import cv2
4 import pathlib
5 # import skimage
6 import scipy
7 import computer_vision
8
9 import matplotlib.pyplot as plt
10 import matplotlib as mpl
11 import numpy as np
12 from functools import reduce
13
14 import torch
15 from models.matching import Matching
16 from models.utils import (AverageTimer, VideoStreamer,
17                             make_matching_plot_fast, frame2tensor)
18
19 torch.set_grad_enabled(False)
20
21
22 def read_image(subdir, image_name):
23     """Reads images from the subdir"""
24     path = pathlib.Path(computer_vision.__path__[0])
25     image_path = path.parents[0] / 'images' / subdir / image_name
26     img_bgr = cv2.imread(image_path)
27     # Convert the image from BGR to RGB format
28     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
29     return img_rgb
30
31 def save_image(image, subdir, image_name):
32     """Saves image to the subdir"""
33     path = pathlib.Path(computer_vision.__path__[0])
34     image_path = path.parents[0] / 'images' / subdir / image_name
35     # Save the image with specific quality
36     img_bgr_for_saving = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
37     cv2.imwrite(image_path, img_bgr_for_saving)
38
39 def save_geometric_distance_output(data):
40     filepath = os.path.join(os.getcwd(), 'geometric_distances.csv')
41     if not os.path.exists(filepath):
42         df = pd.DataFrame(columns=list(data.keys()))
43     else:
44         df = pd.read_csv(filepath)
45         df = df.reset_index(drop=True)
46     # New data as a dictionary
47     new_df = pd.DataFrame([data])
48
49     # Append new data using pd.concat
50     df = pd.concat([df, new_df], ignore_index=True)
51
52     df.to_csv(filepath, index=False)
53     print(f"distances saved to {filepath}")
54
55 def save_pairwise_matches_and_compute_geometric_distances(
56     hw_dir, task_id, n=5, sigma=1.2, distance_threshold = 0.5, use_SIFT=
57     True
58 ):
59     """Saves pairwise matches and computes geometric distances"""
60     # Read images from the subdir
61     img1 = read_image(subdir, image_name1)
62     img2 = read_image(subdir, image_name2)
63     # Compute pairwise matches
64     matches = Matching(img1, img2, n=n, sigma=sigma)
65     # Compute geometric distances
66     distances = Matching.compute_geometric_distances(matches, distance_threshold=distance_threshold)
67     # Save the results
68     save_geometric_distance_output(distances)
69     save_image(img1, subdir, image_name1)
70     save_image(img2, subdir, image_name2)
71     return matches, distances
```

```

58     """This function computes pairwise homographies and saves the
59     images alongwith inlier and outlier matches displayed on top.
60     """
61     if task_id ==1:
62         task_offset = 0
63     elif task_id ==2:
64         task_offset = 10
65
66
67     images = [read_image(subdir=hw_dir, image_name=f'{task_offset+i}.jpg', )
68               for i in range(1, 6)]
69     for i in range(len(images)-1):
70
71         estimator = HomographyEstimator(
72             images[i], images[i+1], distance_threshold=distance_threshold,
73             use_SIFT=use_SIFT)
74
75         H, matches, geometric_distance = estimator.
76             automatic_homography_estimation(
77                 n, sigma=sigma, precondition=True, use_lm=True, use_scipy=False
78             )
79         distances = {
80             'task_id': task_id,
81             'image_pairs': f"{task_offset+i+1}--{task_offset+i+2}",
82             'distance_without_LM': geometric_distance[0],
83             'distance_with_LM': geometric_distance[1],
84         }
85         save_geometric_distance_output(distances)
86
87         inlier_set, outlier_set = matches
88         img_with_matches = display_corresponding_points(
89             images[i], images[i+1], inlier_set, outlier_set[:20], thickness
90             =2
91         )
92         filename = f'combined_{task_offset+i+1}_{task_offset+i+2}.png'
93         save_image(img_with_matches, 'hw5', filename)
94
95
96 def display_corresponding_points(
97     img1, img2, inlier_set, outlier_set,
98     thickness = 2,
99 ):
100     img_combined = np.concatenate([img1, img2], axis=1)
101     # overlay inliers in green...
102     color = color = (0, 255, 0)
103     img_combined = overlay_correspondences(img_combined, inlier_set, img1.
104         shape[1], color=color, thickness=thickness)
105
106     # overlay outliers in red...
107     color = color = (255, 0, 0)
108     img_combined = overlay_correspondences(img_combined, outlier_set, img1.
109         shape[1], color=color, thickness=thickness)
110
111     plt.imshow(img_combined)
112     return img_combined
113
114 def overlay_correspondences(img_combined, corresponding_points, img1_width,
115     color=None, thickness=2):
116     """Given the combined image and correspondences, overlays

```

```

110     the correspondences on the image.
111     """
112     # half = corresponding_points['img1'].shape[0]
113     for ind, correspondence in enumerate(corresponding_points):
114         x, y = correspondence[0]
115         x_p, y_p = correspondence[1]
116         point1 = (int(x), int(y))
117         point2 = (int(img1_width+x_p), int(y_p))
118
119         if color is None:
120             # Randomly generate RGB values (each ranging from 0 to 255)
121             color = (np.random.randint(0, 255), np.random.randint(0, 255),
122                     np.random.randint(0, 255))
123         cv2.line(
124             img_combined, point1, point2, color=color, thickness=thickness
125         )
126     return img_combined
127
128
129 class HomographyEstimator:
130     """Implements automatic Homography estimation pipeline.
131     Steps of the pipeline are;
132     - key points extraction
133     - Finding correspondences (matches)
134     - RANSAC for homography estimation using Linear Least-Square
135       and outlier rejection.
136     - Homography refinement using Non-linear Least-Square (LM)
137
138     """
139     def __init__(self, img1, img2, distance_threshold=0.75, use_SIFT=True)
140     -> None:
141         """Creates estimator for automatic homographic estimation.
142
143         Args:
144             img1: domain image
145             img2: range image
146             distance_threshold: threshold to filter the keypoint matches,
147                               used for ratio test.
148             use_SIFT: If True, use SIFT for extracting key points and
149                      matches, otherwise use SuperPoint and SuperGlue.
150
151         """
152         self.img1 = img1
153         self.img2 = img2
154         self.distance_threshold = distance_threshold
155         self.use_SIFT = use_SIFT
156         self.kp1, self.kp2, self.matches = self.get_key_points()
157         self.total_samples = len(self.matches)
158
159         self.all_data_points = self.get_all_correspondences()
160         self.T, self.T_prime = self.get_pre_conditioning_matrices()
161
162     def get_superGlue_matches(self, img1, img2):
163         device = 'cuda' if torch.cuda.is_available() else 'cpu'
164         config = {
165             'superpoint': {
166                 'nms_radius': 8,
167                 'keypoint_threshold': 0.005,

```

```

167         'max_keypoints': -1
168     },
169     'superglue': {
170         'weights': 'outdoor',
171         'sinkhorn_iterations': 20,
172         'match_threshold': 0.2,
173     }
174 }
175 matching = Matching(config).eval().to(device)
176 data = {
177     'image0': frame2tensor(img1, device),
178     'image1': frame2tensor(img2, device),
179 }
180
181 results = matching(data)
182 kp1, des1 = results['keypoints0'][0].numpy(), results['descriptors0']
183           [0].numpy().T
184 kp2, des2 = results['keypoints1'][0].numpy(), results['descriptors1']
185           [0].numpy().T
186 return kp1, kp2, des1, des2
187
188 def get_key_points(self):
189     img1_gray = cv2.cvtColor(self.img1, cv2.COLOR_RGB2GRAY)
190     img2_gray = cv2.cvtColor(self.img2, cv2.COLOR_RGB2GRAY)
191
192     if self.use_SIFT:
193         sift = cv2.SIFT_create()
194         kp1, des1 = sift.detectAndCompute(img1_gray, None)
195         kp2, des2 = sift.detectAndCompute(img2_gray, None)
196     else:
197         kp1, kp2, des1, des2 = self.get_superGlue_matches(img1_gray,
198                                                         img2_gray)
199
200     bf = cv2.BFMatcher()
201     matches = bf.knnMatch(des1, des2, k=2)
202
203     # Apply ratio test
204     retained_matches = []
205     for m, n in matches:
206         if m.distance < self.distance_threshold * n.distance: # Ratio
207             threshold, can tweak this value
208             retained_matches.append(m)
209
210     return kp1, kp2, retained_matches
211
212 def get_all_correspondences(self):
213     """Returns all point correspondences.
214
215     Returns:
216         (N, 2, 2): (num_corres, point, coordinates)
217
218     """
219     pt_correspondences = []
220     for id in range(self.total_samples):
221         corresp = self.matches[id]
222         x = self.kp1[corresp.queryIdx]

```

```

222         x_prime = self.kp2[corresp.trainIdx]
223         if self.use_SIFT:
224             x = x.pt
225             x_prime = x_prime.pt
226
227         pt_correspondences.append((x, x_prime))
228         pt_correspondences = np.array(pt_correspondences)
229         return pt_correspondences
230
231
232     def sample_data_points(self, n):
233         """Randomly select 'n' samples from the full data points."""
234         random_ids = np.random.randint(0, self.total_samples, n)
235         return self.all_data_points[random_ids]
236
237
238
239     def get_pre_conditioning_matrices(self):
240         """Isotropic scalling to pre-condition points."""
241         img1_h, img1_w, _ = self.img1.shape
242         pixels_x, pixels_y = np.meshgrid(np.arange(img1_w), np.arange(img1_h))
243         dom_pixels = np.stack([pixels_x.flatten(), pixels_y.flatten()], axis=1)
244
245         T = HomographyEstimator.compute_pre_conditioning_transform(dom_pixels)
246
247         img2_h, img2_w, _ = self.img2.shape
248         pixels_x, pixels_y = np.meshgrid(np.arange(img2_w), np.arange(img2_h))
249         range_pixels = np.stack([pixels_x.flatten(), pixels_y.flatten()], axis=1)
250
251         T_prime = HomographyEstimator.compute_pre_conditioning_transform(range_pixels)
252         return T, T_prime
253
254
255     @staticmethod
256     def compute_pre_conditioning_transform(points_2d):
257         # compute centroid
258         centroid = np.mean(points_2d, axis=0)
259         # avg distance of point from centroid
260         d = np.sqrt(np.sum((points_2d - centroid[None, :])**2, axis=1))
261         d_avg = np.mean(d)
262
263         # scale factor to set avg distance to sqrt(2)
264         s = np.sqrt(2) / d_avg
265
266         # pre-conditioning matrix
267         T = np.eye(3)
268         T[0,0] = s
269         T[1,1] = s
270         T[:2, -1] = -s*centroid
271         return T
272
273
274     @staticmethod

```



```

275 def pre_condition_points(points_2d, T):
276     """Isotropic scalling to pre-condition points."""
277
278     points_hc = np.concat([points_2d, np.ones(points_2d.shape[0])[:,None
279                               ]], axis=1)
280     points_norm = np.matmul(T, points_hc.T)
281     return points_norm.T
282
283 @staticmethod
284 def get_coefficients_of_linear_equations(x, x_prime):
285     """Return the coefficients of linear equations,
286     for the given correspondence.
287
288     Args:
289         x = (3,)
290         x_prime = (3,)
291     """
292     return np.array([[0, 0, 0, -x_prime[2]*x[0], -x_prime[2]*x[1], -
293                       x_prime[2]*x[2], x_prime[1]*x[0], x_prime[1]*x[1], x_prime[1]*x
294                       [2]],
295                     [x_prime[2]*x[0], x_prime[2]*x[1], x_prime[2]*x[2], 0, 0, 0,
296                       -x_prime[0]*x[0], -x_prime[0]*x[1], -x_prime[0]*x[2]])
297
298 def get_coefficient_matrix(self, dom_pts, range_pts):
299     """For the given data points, get the coefficients matrix
300     A for homogeneous system of equations.
301
302     Args:
303         dom_pts: (N, 3)
304         range_pts: (N, 3)
305     """
306     A = []
307     for dom_pt, range_pt in zip(dom_pts, range_pts):
308         A_i = HomographyEstimator.get_coefficients_of_linear_equations(
309             dom_pt, range_pt)
310         A.append(A_i)
311     return np.concat(A, axis=0)
312
313 @staticmethod
314 def lst_square(A):
315     """Returns linear least square solution to homogeneous
316     system of linear equations,  $Ah = 0$ .
317     Since Rank(A) = n-1, solution is the eigen vector
318     corresponding to the smallest singular value of A
319     i.e. last row of vh.
320     """
321     u, s, vh = np.linalg.svd(A)
322     return vh[-1]
323
324 def fit(self, data_points, precondition=False):
325     """For the given data points, compute homography matrix
326     using linear least square fit.
327
328     Args:
329         data_points: (n, 2, 2) = n correspondences,
330                     [:,0,:] domain pts,[:,1,:] range pts
331     """

```

```

329     # print(data_points.shape)
330     dom_pts = data_points[:,0]
331     range_pts = data_points[:,1]
332     if precondition:
333         dom_pts = HomographyEstimator.pre_condition_points(dom_pts, self
334             .T)
335         range_pts = HomographyEstimator.pre_condition_points(range_pts,
336             self.T_prime)
337         # A = self.get_coefficient_matrix(dom_pts_norm, range_pts_norm)
338     else:
339         dom_pts = np.concat([dom_pts, np.ones(dom_pts.shape[0])[:, None
340             ]], axis=1)
341         range_pts = np.concat([range_pts, np.ones(range_pts.shape[0])[:,
342             None]], axis=1)
343
344     A = self.get_coefficient_matrix(dom_pts, range_pts)
345     h = HomographyEstimator.lst_square(A)
346
347     h /= h[-1]
348     H = h.reshape(3,3)
349     if precondition:
350         H_tilde = np.matmul(H, self.T)
351         H = np.matmul(np.linalg.pinv(self.T_prime), H_tilde)
352         H /= H[-1,-1]
353
354     return H
355
356 @staticmethod
357 def apply_homography(H, physical_pts):
358     """Apply homography matrix H, to physical points.
359     Args:
360         H= (3 x 3) homography matrix
361         physical_pts= (N, 2) physical coordinates
362     """
363     hc_pts = np.concat([physical_pts, np.ones(physical_pts.shape[0])[:,
364         None]], axis=1).transpose()
365     hc_pts_prime = np.matmul(H, hc_pts)
366     hc_pts_prime /= hc_pts_prime[-1]
367     return hc_pts_prime[:2].transpose()
368
369 def get_inlier_set(self, H, inlier_threshold):
370     """Returns the inlier set for the given homography H,
371     and threshold.
372
373     Returns:
374         inlier_set:
375         agg_distance: total distance for the inlier set.
376     """
377
378     dom_pts = self.all_data_points[:,0]
379     range_pts = self.all_data_points[:,1]
380
381     # deviation in range space..
382     pred_range_pts = HomographyEstimator.apply_homography(H, dom_pts)
383     d1 = np.sum((range_pts - pred_range_pts)**2, axis=1)
384
385     # deviation in dom space..
386     H_inv = np.linalg.pinv(H)

```

```

382     pred_dom_pts = HomographyEstimator.apply_homography(H_inv, range_pts
383     )
384     d2 = np.sum((dom_pts - pred_dom_pts)**2, axis=1)
385
386     dev = np.sqrt(d1 + d2)
387     inlier_mask = np.where(dev < inlier_threshold)[0]
388     # print(f"mask shape: {inlier_mask.shape}")
389
390     inlier_set = self.all_data_points[inlier_mask]
391     outlier_set = self.all_data_points[~inlier_mask]
392     agg_distance = np.sum(dev[inlier_mask])
393     return inlier_set, agg_distance, outlier_set
394
395 @staticmethod
396 def cost_fn(h, point_correspondences):
397     """computes the cost for the given point correspondences
398     and parameter h.
399     """
400     residuals = HomographyEstimator.compute_residuals(h,
401     point_correspondences)
402     return np.sum(residuals**2)
403
404 @staticmethod
405 def compute_residuals(h, point_correspondences):
406     """computes the residuals for the given point correspondences
407     and parameter h.
408     """
409     residuals = []
410     for corresp in point_correspondences:
411         x = corresp[0]
412         x_prime = corresp[1]
413         denom = h[6]*x[0] + h[7]*x[1] + 1
414
415         res_0 = x_prime[0] - (h[0]*x[0] + h[1]*x[1] + h[2])/denom
416         res_1 = x_prime[1] - (h[3]*x[0] + h[4]*x[1] + h[5])/denom
417         residuals.append(res_0)
418         residuals.append(res_1)
419     return np.array(residuals)
420
421 @staticmethod
422 def jacobian_f(h, point_correspondences):
423     """computes the cost for the given point correspondences
424     and parameter h.
425     """
426     jacobians = []
427     for corresp in point_correspondences:
428         x = corresp[0]
429         numer1 = h[0]*x[0] + h[1]*x[1] + h[2]
430         numer2 = h[3]*x[0] + h[4]*x[1] + h[5]
431         denom = h[6]*x[0] + h[7]*x[1] + 1
432         jac_f1 = [x[0]/denom, x[1]/denom, 1/denom, 0, 0, 0, -x[0]*numer1
433         /(denom**2), -x[1]*numer1/(denom**2)]
434         jac_f2 = [0, 0, 0, x[0]/denom, x[1]/denom, 1/denom, -x[0]*numer2
435         /(denom**2), -x[1]*numer2/(denom**2)]
436         jacobians.append(jac_f1)
437         jacobians.append(jac_f2)
438     return np.array(jacobians)

```

```

437
438 def RANSAC(self, n, sigma, p=0.99, precondition=True):
439     """
440
441     Args:
442         n = number of data points to fit the estimate.
443         sigma = std. deviation of inlier noise
444         epsilon = probability of outlier.
445         p = probability that at least one trial will be free of outliers
446
447     """
448     epsilon = 0.90 # starting with worst-case guess..
449     n_total = self.total_samples
450     N = np.log(1 - p)/np.log(1 - (1 - epsilon)**n)
451     M = (1 - epsilon)*n_total
452     inlier_threshold = 3*sigma
453     print(f"Total samples: {n_total}")
454     print(f"Acceptable size of inlier set: {M}")
455     print(f"Number of iterations: {N}")
456     print(f"Inlier threshold: {inlier_threshold:.2f}")
457
458     bestErr = None
459     bestInSet = None
460     sizeOfInset = 0
461     outSet = None
462     bestFit = None
463     itr_count = 0
464     while N > itr_count:
465         data_points = self.sample_data_points(n)
466         model = self.fit(data_points, precondition=precondition)
467         inlier_set, modelErr, outlier_set = self.get_inlier_set(model,
468             inlier_threshold)
469         print(f"itr: {itr_count}, inlier set: {inlier_set.shape[0]},
470             modelErr: {modelErr}")
471         # print(f"itr: {i}, modelErr: {modelErr}")
472         if inlier_set.shape[0] > sizeOfInset:
473             # bigger inset is found..
474             bestFit = model
475             bestErr = modelErr
476             bestInSet = inlier_set
477             outSet = outlier_set
478             sizeOfInset = inlier_set.shape[0]
479
480             # update estimate of iterations...
481             epsilon = 1 - inlier_set.shape[0]/n_total
482             N = np.log(1 - p)/np.log(1 - (1 - epsilon)**n)
483             M = (1 - epsilon)*n_total
484             print(f"epsilon: {epsilon:.2f}")
485             print(f"Estimate of required iterations: {N}")
486         elif bestErr is not None and inlier_set.shape[0] == sizeOfInset
487             and modelErr < bestErr:
488             # better inset of the same size is found..
489             bestFit = model
490             bestErr = modelErr
491             bestInSet = inlier_set
492             outSet = outlier_set
493
494     itr_count += 1

```

```

492         if bestFit is not None:
493             # Now fit using the entire inlier_set, to refine the bestFit
494             bestFit = self.fit(bestInSet, precondition=precondition)
495             bestInSet, bestErr, outSet = self.get_inlier_set(bestFit,
496                 inlier_threshold)
497             bestFit = self.fit(bestInSet, precondition=precondition)
498         return bestFit, bestInSet, outSet, bestErr
499
500     @staticmethod
501     def get_refined_homography(H, inlier_set, use_scipy=False, t=0.01,
502         convergence_th=0.01, max_itr=50):
503         """Refine homography estimate using non-linear least squares
504         for the correspondences from the inlier set.
505
506         Args:
507             H: starting H, (initial guess)
508             inlier_set: set of (noisy) correspondences
509
510         """
511         if use_scipy:
512             h = scipy.optimize.least_squares(
513                 HomographyEstimator.compute_residuals, H.reshape(-1)
514                [:-1], method='lm', args=(inlier_set, )
515             ).x
516         else:
517             h = HomographyEstimator.optimize_using_LM(
518                 H.reshape(-1)[:-1], inlier_set, t=t, convergence_th=
519                 convergence_th,
520                 max_itr=max_itr
521             )
522             H = np.concat([h, np.ones(1)]).reshape(3,3)
523         return H
524
525     @staticmethod
526     def optimize_using_LM(h, inlier_set, t=0.01, convergence_th=0.01,
527         max_itr=50):
528
529         J = HomographyEstimator.jacobian_f(h, inlier_set)
530         mu = t*max(np.diag(np.matmul(J.transpose(), J)))
531
532         for itr in range(max_itr):
533             J = HomographyEstimator.jacobian_f(h, inlier_set)
534             E = HomographyEstimator.compute_residuals(h, inlier_set)
535
536             C_p = HomographyEstimator.cost_fn(h, inlier_set)
537             delta_h = np.linalg.pinv(J.T@J + mu*np.eye(h.size))@J.T @ E
538             C_p1 = HomographyEstimator.cost_fn(h + delta_h, inlier_set)
539
540             rho = (C_p - C_p1) / (delta_h.T @J.T @ E + delta_h.T @ (mu*np.
541                 eye(h.size)) @ delta_h)
542             # mu update
543             mu = mu*max(1/3, 1- (2*rho -1)**3)
544             h = h + delta_h
545             if np.linalg.norm(delta_h, 1) < convergence_th:
546                 break
547
548         print(f"H converged in {itr+1} steps")
549         return h
550
551     def automatic_homography_estimation(

```

```

545         self, n, sigma, precondition=True, use_lm=True, use_scipy=False,
546             convergence_th=0.01):
547         """Automatically estimates homography"""
548
549         p = 0.99
550         H, inlier_set, outlier_set, model_error = self.RANSAC(n, sigma, p=p,
551             precondition=precondition)
552
553         initial_geometric_distance = HomographyEstimator.cost_fn(H.reshape
554             (-1)[: -1], inlier_set)
555         if use_lm:
556             H = HomographyEstimator.get_refined_homography(
557                 H, inlier_set, use_scipy=use_scipy, convergence_th=
558                     convergence_th
559             )
560         improved_geometric_distance = HomographyEstimator.cost_fn(H.reshape
561             (-1)[: -1], inlier_set)
562
563         return H, (inlier_set, outlier_set), (initial_geometric_distance,
564             improved_geometric_distance)
565
566 class Panorama:
567     def __init__(
568         self, images, homography_config, ref_img_index=None, max_size
569             =4000,
570     ) -> None:
571         self.images = images
572         self.num_images = len(images)
573         if ref_img_index is None:
574             ref_img_index = len(images)//2
575         self.ref_index = ref_img_index
576
577         self.max_size = max_size
578         self.homography_config = homography_config
579
580     def get_combined_H_ref(self):
581
582         H_mats_pairwise = self.compute_pairwise_homographies()
583         H_mats_ref = []
584         for i, img in enumerate(self.images):
585             H = self.get_H_to_ref_frame(i, H_mats_pairwise)
586             H_mats_ref.append(H)
587
588         return H_mats_ref
589
590     def get_H_to_ref_frame(self, index, homography_matrices):
591         """
592         Computes a homography matrix to transform an image at 'index' to the
593             'ref_frame_index'.
594
595         Args:
596             index (int): The index of the image.
597             ref_frame_index (int): The reference frame index.
598             homography_matrices (list of np.ndarray): Pairwise homographies
599                 between consecutive images.

```

```

595     Returns:
596         np.ndarray: The combined homography matrix to map the image at '
597             index' to the reference frame.
598     """
599     H_list = []
600     ref_index = self.ref_index
601     while index != ref_index:
602         if index < ref_index:
603             H_list = [homography_matrices[index]] + H_list
604             index += 1
605         elif index > ref_index:
606             H_list.append(np.linalg.pinv(homography_matrices[index-1]))
607             index -= 1
608     if len(H_list) > 0:
609         combined_H = reduce(np.matmul, H_list)
610     else:
611         combined_H = np.eye(3)
612     return combined_H
613
614 def compute_pairwise_homographies(self):
615     """
616     Computes pairwise homography matrices between consecutive images.
617
618     Returns:
619         list of np.ndarray: Pairwise homography matrices between
620             consecutive images.
621     """
622     n = self.homography_config.pop('n', 5)
623     sigma = self.homography_config.pop('sigma', 2)
624     distance_threshold = self.homography_config.pop('distance_threshold',
625         0.75)
626     use_SIFT = self.homography_config.pop('use_SIFT', True)
627     precondition = self.homography_config.pop('precondition', True)
628     use_lm = self.homography_config.pop('use_lm', True)
629     use_scipy = self.homography_config.pop('use_scipy', False)
630
631     H_mats_pairwise = []
632
633     for i in range(self.num_images-1):
634         estimator = HomographyEstimator(
635             self.images[i], self.images[i+1], distance_threshold=
636                 distance_threshold,
637                 use_SIFT=use_SIFT
638         )
639         H, matches, geometric_distance = estimator.
640             automatic_homography_estimation(
641                 n, sigma=sigma, precondition=precondition, use_lm=use_lm,
642                 use_scipy=use_scipy,
643             )
644         H_mats_pairwise.append(H)
645     return H_mats_pairwise
646
647 @staticmethod
648 def get_corner_points(H, img):

```



```

649     """Get the corners of the transformed img."""
650     h,w,_ = img.shape
651     corners = np.array([[0,0], [w, 0], [w, h], [0, h]])
652     corners_hc = np.concat([corners, np.ones(corners.shape[0])[:,None]],
        axis=1)
653
654     corners_tr = np.matmul(H, corners_hc.T)
655     corners_tr /= corners_tr[-1]
656
657     x_min, y_min = np.min(corners_tr[:-1], axis=1)
658     x_max, y_max = np.max(corners_tr[:-1], axis=1)
659     return x_min, y_min, x_max, y_max
660
661
662     def get_canvas_dims(self, H_mats):
663
664         ref_img = self.images[self.ref_index]
665         ref_h, ref_w, _ = ref_img.shape
666         canvas_x_min, canvas_x_max, canvas_y_min, canvas_y_max = 0, ref_w,
        0, ref_h
667         for i, img in enumerate(self.images):
668             x_min, y_min, x_max, y_max= Panorama.get_corner_points(H_mats[i
        ], img=img)
669
670             if x_min < canvas_x_min:
671                 canvas_x_min = x_min
672             if y_min < canvas_y_min:
673                 canvas_y_min = y_min
674             if x_max > canvas_x_max:
675                 canvas_x_max = x_max
676             if y_max > canvas_y_max:
677                 canvas_y_max = y_max
678
679         canvas_h = canvas_y_max - canvas_y_min
680         canvas_w = canvas_x_max - canvas_x_min
681
682         # adjust origin for unequal stretching on both sides of origin...
683         offset_x = (abs(canvas_x_max) - abs(canvas_x_min))/2
684         offset_y = (abs(canvas_y_max) - abs(canvas_y_min))/2
685         origin_x = canvas_w/2 - offset_x
686         origin_y = canvas_h/2 - offset_y
687
688         scale_factor = max(max(canvas_h/self.max_size, canvas_w/self.
        max_size), 1)
689         canvas_h /= scale_factor
690         canvas_w /= scale_factor
691         if scale_factor > 1:
692             print(f"Output canvas dim adjusted to have max dim of {self.
        max_size}")
693         return (int(canvas_h), int(canvas_w)), (int(origin_x), int(origin_y)
        ), scale_factor
694
695
696     def get_panorama_view(self):
697
698
699         H_mats_ref = self.get_combined_H_ref()
700         canvas_dims, origin, scale_factor = self.get_canvas_dims(H_mats_ref)
701

```

```

702     canvas = np.zeros((*canvas_dims, 3), dtype=np.uint)
703
704     for i, img in enumerate(self.images):
705         if i != self.ref_index:
706             canvas = self.project_img_onto_canvas(H_mats_ref[i], img,
707                                                  canvas, origin, scale_factor)
708         i = self.ref_index
709         canvas = self.project_img_onto_canvas(H_mats_ref[i], self.images[i],
710                                              canvas, origin, scale_factor)
711     return canvas.astype(np.uint8)
712
713 def project_img_onto_canvas(self, H, img, canvas, origin, scale_factor):
714     """
715     Projects an image onto a canvas using a homography matrix with
716     bilinear interpolation.
717
718     Args:
719         H (np.ndarray): Homography matrix.
720         img (np.ndarray): Image to project.
721         canvas (np.ndarray): Canvas to project onto.
722         origin (tuple): Origin coordinates for placement on the canvas.
723         scale_factor (float): Scaling factor for transformation.
724
725     Returns:
726         None. The canvas is modified in-place.
727     """
728     # Adjust homography matrix
729     H = Panorama.adjust_H_for_output_size(H, origin, scale_factor)
730     canvas_h, canvas_w, _ = canvas.shape
731     # img_height, img_width, _ = img.shape
732
733     # Inverse homography
734     H_inv = np.linalg.pinv(H)
735
736     # Create homogeneous coordinates for canvas pixels
737     pixel_y, pixel_x = np.meshgrid(range(canvas_h), range(canvas_w))
738     hc_coordinates = np.stack([pixel_x.flatten(), pixel_y.flatten(), np.
739                               ones_like(pixel_x).flatten()])
740
741     # Transform canvas coordinates to image space
742     transformed_coordinates = np.matmul(H_inv, hc_coordinates)
743     transformed_coordinates /= transformed_coordinates[-1]
744
745     img_values, mask_coordinates = Panorama.bilinear_interpolation(img,
746                                                                    transformed_coordinates[:-1])
747     # Project onto canvas
748     canvas[pixel_y.flatten()[mask_coordinates], pixel_x.flatten()[
749            mask_coordinates]] = img_values[mask_coordinates]
750     return canvas.astype(np.uint8)
751
752 @staticmethod
753 def bilinear_interpolation(img, coordinates):
754     """For the given img and coordinates, uses bilinear interpolation
755     and returns img values for the input coordinates.
756
757     Args:
758         transformed_coordinates = (2, n)
759     """
760     img_height, img_width, _ = img.shape

```

```

755     # Extract integer and fractional parts of coordinates
756     x_trans = coordinates[0]
757     y_trans = coordinates[1]
758     x_floor = np.floor(x_trans).astype(int)
759     y_floor = np.floor(y_trans).astype(int)
760     x_ceil = np.ceil(x_trans).astype(int)
761     y_ceil = np.ceil(y_trans).astype(int)
762
763     # Clip coordinates to image boundaries
764     x_floor = np.clip(x_floor, 0, img_width - 1)
765     y_floor = np.clip(y_floor, 0, img_height - 1)
766     x_ceil = np.clip(x_ceil, 0, img_width - 1)
767     y_ceil = np.clip(y_ceil, 0, img_height - 1)
768
769     # Bilinear interpolation weights
770     wx = x_trans - x_floor # fractional part in x
771     wy = y_trans - y_floor # fractional part in y
772
773     # Perform bilinear interpolation
774     img_values = ((1 - wx) * (1 - wy))[:, None] * img[y_floor, x_floor]
775     + \
776     (wx * (1 - wy))[:, None] * img[y_floor, x_ceil] + \
777     ((1 - wx) * wy)[:, None] * img[y_ceil, x_floor] + \
778     (wx * wy)[:, None] * img[y_ceil, x_ceil]
779     # Create a mask for valid (within image bounds) coordinates
780     mask_coordinates = (x_trans >= 0) & (x_trans < img_width) & \
781     (y_trans >= 0) & (y_trans < img_height)
782     return img_values, mask_coordinates
783
784
785
786 @staticmethod
787 def adjust_H_for_output_size(H, origin, scale_factor):
788     """Adjust the homography matrix to set the desired output size
789     and origin.
790
791     Args:
792         H: Homography matrix H
793         origin: desired origin (in the projected output size space).
794         scale_factor: specifies adjusted to proj size.
795     """
796     T = np.eye(3)
797     T[:2, -1] = origin[0], origin[1]
798     H = np.matmul(T, H)
799
800     H_resize = np.eye(3)
801     H_resize[0,0] = 1/scale_factor
802     H_resize[1,1] = 1/scale_factor
803     H = np.matmul(H_resize, H)
804     return H
805
806
807
808
809
810 if __name__ == '__main__':
811

```

```

812     geometric_distances_file = os.path.join(os.getcwd(), '
      geometric_distances.csv')
813     if os.path.exists(geometric_distances_file):
814         os.remove(geometric_distances_file)
815
816     # Config for mosaic image.
817     homography_config = {
818         'n': 5,
819         'sigma': 1.2,
820         'distance_threshold': 0.75,
821         'use_SIFT': True,
822         'precondition': True,
823         'use_lm': True,
824         'use_scipy': False
825     }
826     max_size = 6000
827
828     for task_id in [1,2]:
829         # Task-1
830         # task_id = 1
831         hw_dir = 'hw5'
832         n=5
833         sigma=1.2
834         distance_threshold = 0.5
835         use_SIFT=True
836         save_pairwise_matches_and_compute_geometric_distances(
837             hw_dir=hw_dir, task_id=task_id, n=n, sigma=sigma,
838             distance_threshold=distance_threshold,
839             use_SIFT=use_SIFT
840         )
841
842         if task_id ==1:
843             task_offset = 0
844         elif task_id ==2:
845             task_offset = 10
846         images = [read_image(subdir=hw_dir, image_name=f'{task_offset+i}.jpg'
847             , ) for i in range(1, 6)]
848
849         homography_config['use_lm'] = True
850         panorama = Panorama(images, homography_config=homography_config,
851             max_size=max_size)
852         img_mosaic = panorama.get_panorama_view()
853
854         filename = f'mosaic_image_task{task_id}_using_lm.png'
855         save_image(img_mosaic, 'hw5', filename)
856
857         homography_config['use_lm'] = False
858         panorama = Panorama(images, homography_config=homography_config,
859             max_size=max_size)
860         img_mosaic = panorama.get_panorama_view()
861
862         filename = f'mosaic_image_task{task_id}_without_lm.png'
863         save_image(img_mosaic, 'hw5', filename)

```

Listing 1: Example Python code